Indoor navigation among walking people: a measured pipeline in Isaac Sim

Dimitrios Gkiokas · dimitris.gkiokas@outlook.com · July 2026

Abstract. I built a simulation benchmark that takes one robot from reactive wandering to goal-directed autonomy in stages, and measures each stage against simulator ground truth. Worlds are generated from seeds: four rooms, doors of controlled width, furniture, and people who walk patrol routes through doorways. Across 25 worlds per condition the pipeline produced: a door-traversal success curve with its transition where body geometry predicts it, 75 paired runs showing that walking people cost interaction pressure but no exploration speed, 25 maps built autonomously under human motion (median 63% wall coverage in 20 minutes), and a paired navigation experiment in which an offline reachability check promised 85 of 100 goals while the live stack truly reached 11, with localization failure explaining the gap. Retuning the localizer removed every false arrival (6 to 0) and changed the true count by one. All code, data, and analysis are public.

1. Introduction

The question behind this project is practical: what does it take for a small wheeled robot to move with purpose through cluttered rooms while people walk around it? Rather than answer with one demonstration, I broke the problem into stages that each produce a number: how narrow a door the reactive layer can pass, what human traffic costs, how good a map the robot can build for itself under that traffic, and how far a standard navigation stack gets on that self-built map. The stages share one discipline. Every result is checked against the simulator's ground truth, never against the robot's own belief. That choice ends up carrying the main finding of the last experiment, where the stack's self-reported successes and its actual positions disagree by meters.

The pipeline runs in NVIDIA Isaac Sim 6.0 on Windows with ROS 2 Jazzy in WSL2, on one RTX 2080 Ti, below the simulator's minimum specification. Real-time factors sat between 0.35 and 0.55 for the whole campaign. I mention the hardware because the engineering that keeps a two-machine, two-clock pipeline honest at half speed is part of the work: wall-clock stamps everywhere, file-based handshakes between the simulator and the ROS side, and batch machinery that survived 50-plus unattended world regenerations.

2. The benchmark factory

A world is a function of a seed. The generator partitions a 20 × 20 m arena into four rooms with internal partition walls, cuts doors whose widths are drawn from two bands (one anchor door per wall at 0.72–0.95 m, harder doors at 0.50–0.78 m), scatters furniture, and writes a manifest CSV: every wall, door, and obstacle with coordinates, plus patrol routes for up to three people. The manifest is the ground truth that every later measurement joins against. People walk their routes at constant speed with per-waypoint pauses. Cross-room walkers are routed through the widest door of their wall, which turns doorways into contested resources.

The robot is a 0.42 m differential-drive body. Its lidar is a 360-ray PhysX raycast published to /scan at about 20 Hz. Odometry integrates ground-truth motion with multiplicative noise (3% linear, 5% angular), which produced measured end-of-run drifts up to 1.57 m. A logger records robot and walker ground-truth poses at 10 Hz into one CSV per run. Batches run unattended: the simulator side regenerates worlds and signals the WSL side through files. A deadlock watchdog ends runs whose full position trail stays inside a 0.5 m box for 180 s. An earlier watchdog compared only the trail's endpoints and killed 16 of 25 healthy runs whose paths looped back near old positions. The post-mortem of that false-kill batch is what fixed the rule.

3. Reactive navigation vs door width

The reactive layer is a follow-the-gap navigator over the raw scan with 13 escalating recovery behaviors, plus a camera-based person brake (YOLOv8n). Its predecessor deadlocked in tight geometry. Version 2 finished 75 consecutive 20-minute runs without a single deadlock. Success at doors is a function of width: near-certain above 0.72 m, near-zero below 0.55 m, with the transition where the 0.42 m body plus sensing margin says it should be (Figure 1). This curve is the reason later worlds carry doors sampled around that boundary: they are the probe.

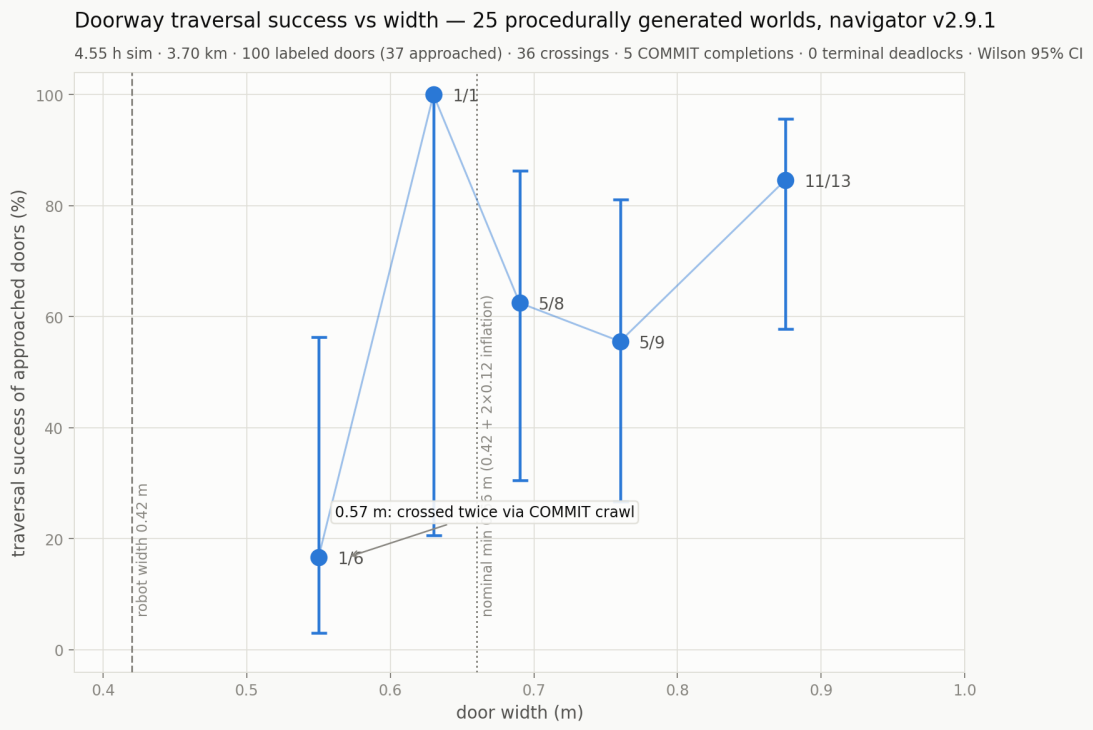

Figure 1. Door traversal success against door width, 25 seeded worlds. Dashed lines mark the 0.42 m body width and the nominal minimum with sensor inflation.

4. What walking people cost

The same 25 worlds ran three times: empty, with two walkers, and with three walkers of which one crosses rooms through a contested door. That is 75 runs, geometry identical per seed (verified by hashing), 20 minutes each. Exploration speed did not move: 13.8, 13.8, and 13.9 m/min. What moved is pressure. Door-conflict episodes rose from 26 to 54 per condition, robot–walker crossings rose to about 10 per hour, and the whole campaign produced exactly one near-contact at 0.27 m. Its anatomy is instructive: a walker emerged through a door mouth inside the camera's blind frontal sector, the person-brake never fired, and the lidar reflex layer absorbed the encounter. The navigator's own logs attribute the extra work cleanly: person-triggered avoidance rose by a factor of 2.2 and trap-escape recoveries by 3.7, while the share of time in plain gap-following stayed at 89.6% in both dynamic conditions.

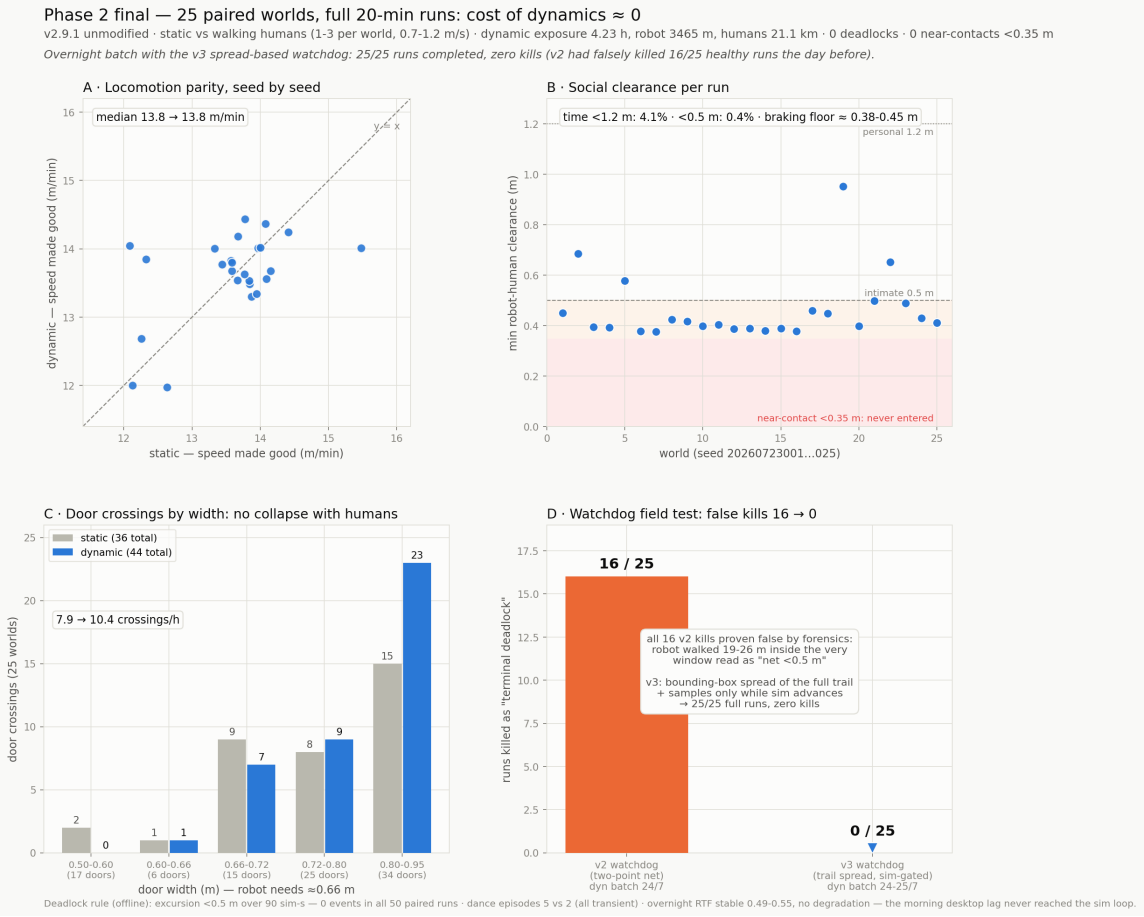

Figure 2. Paired static vs dynamic comparison across 25 worlds: exploration rate unchanged, interaction metrics double.

5. Mapping while people walk

Stage three hands the same wandering robot to slam_toolbox: 25 worlds, 20 minutes each, odometry noise and walkers active, one map saved per world by an orchestrator that restarts SLAM between worlds. All 25 maps saved without intervention overnight. Two results matter later. First, walking people are nearly free: human encounter time does not correlate with coverage ($r = -0.13$), and walkers leave almost no permanent imprint, unlike standing people, which imprint an order of magnitude more occupied cells. Second, the wander policy is the bottleneck: median wall-surface coverage is 63% (range 47–92%), against 82% median occupied-cell precision. The robot maps what it visits accurately. It just does not visit enough. Doors come out narrower on the map than in truth by a median of 5 cm, which matters at a planner's inflation radius. Measured odometry drift correlates mildly with map precision ($r = -0.37$, $n = 25$).

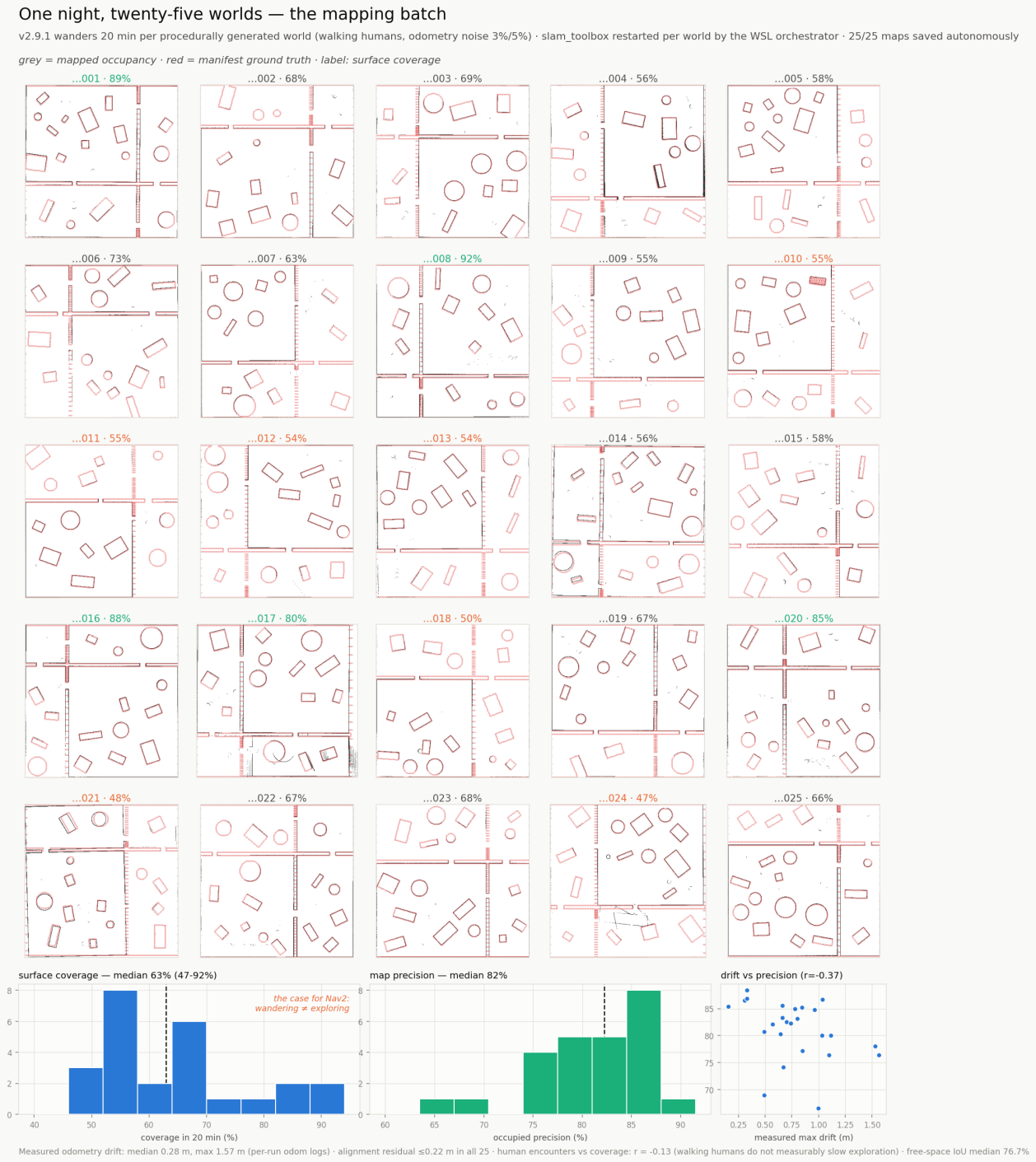

Figure 3. Twenty-five worlds mapped in one night. Grey: mapped occupancy. Red: manifest ground truth. Per-world label: wall-surface coverage.

6. Goal-directed missions on self-built maps

The final stage gives Nav2 (AMCL, NavFn, DWB) each world's self-built map and a four-goal tour: the centers of the three rooms the robot has never been sent to, then home. Before running anything I computed, per world, which goals are reachable at the planner's 0.30 m radius on the map and on the ground-truth geometry (a BFS over the inflated grid). The check says 85 of 100 goals are feasible on the maps, and flags four worlds as cages whose spawn room has no door the conservative planner will accept.

The live runs then measured the distance between promise and delivery. With default AMCL parameters the stack truly reached 11 of 100 goals, where truly means the ground-truth position at the moment of reported success lies within 1.5 m of the goal. Six more goals were reported reached and were not. The worst case reported two arrivals 0.1 s apart while the robot stood at its spawn point, 7 m from either goal: the particle filter's belief had collapsed

onto the goals in a sparsely mapped room. First crossings into adjacent rooms mostly work (7 of 25 worlds, arrivals verified to 0.12–0.29 m). It is the second room, mapped at 50-something percent, where localization dies. The four predicted cage worlds refused every cross-room goal, as they should.

I then retuned AMCL for sparse maps (500–4000 particles, wider motion model, tolerance for beams that the map cannot explain) and re-ran all 25 tours: a paired experiment with one changed variable. The tuning removed every false arrival (6 to 0) and every delirium world (4 to 0). The true count moved from 11 to 12. Failures became honest: worlds that previously hallucinated success now time out or wedge in place, knowing they have not arrived (wedged worlds 4 to 8). The cage worlds refused again, in both configurations, 8 of 8 with the prediction. The reading I defend: a better estimator stops lying, but it cannot recover information the 20-minute map never captured. The ceiling belongs to exploration, not to the filter (Figure 4).

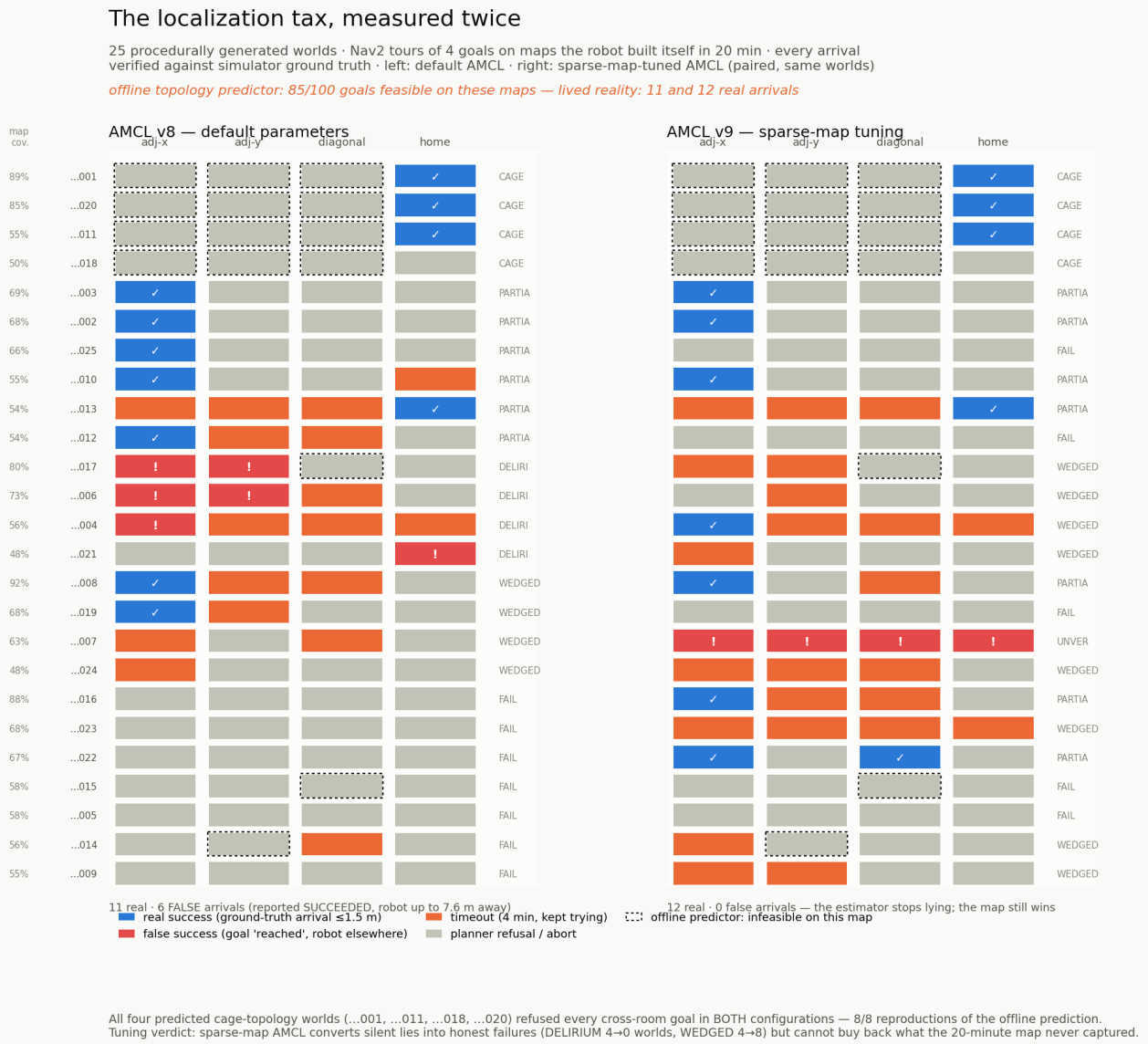

Figure 4. The paired mission experiment: 25 worlds × 4 goals, default AMCL (left) vs sparse-map tuning (right), every cell verified against ground truth. Dotted cells: goals the offline check calls infeasible on that map.

7. What I would tell the next builder

Verify against ground truth from day one. Every major error in this project was caught by a join between logs and the manifest: the watchdog that killed healthy runs, the mapping run that silently used zero noise, the navigation stack that reported success from the wrong room. Self-reported metrics would have let all three through. Second, on a two-machine pipeline, prefer explicit signal files to clocks and file attributes. WSL's attribute cache served stale timestamps and broke a trigger that file existence handled cleanly. Third, long-running ROS 2 batches need hygiene between iterations: shared-memory segments from killed DDS participants accumulate until the command-line tools

stop seeing the graph. Fourth, inside the simulator, cache no handles across stage reloads. Rebinding on every play is what let one session survive fifty world regenerations.

8. Limits

Twenty-five worlds per condition supports the patterns reported here, not fine statistics. The feasibility check is a BFS approximation of the planner, not the planner. The 1.5 m arrival threshold absorbs the timing tolerance of the offline log join. One tour of fifty (world 007, tuned run) returned instant successes with no matching robot motion and is excluded as unverifiable. The exclusion is recorded in the result files. Everything ran in one simulator on one machine, and the wall-clock timing design, which is why the stack tolerates real-time factor swings, does not emulate a real robot's clock discipline.

9. Future work

The mapping stage points at the bottleneck: coverage. Frontier-based exploration in place of reactive wander attacks the ceiling that localizer tuning could not move. On the reactive side, the one near-contact specifies a concrete behavior that does not exist yet: door-mouth anticipation from lidar range-rate. Smaller follow-ups: wheel-encoder odometry in place of the noise model, and a backtrack recovery that retreats along the entry path instead of spinning in place.